UNIVERSITÉ CAEN NORMANDIE

Licence 3 Informatique

Projet de conception logicielle 3 - Jeu de Tron

Équipe de développement :

N°	Prénom et nom	Numéro
1	Lamia **HADJ BENABDELMOULA**	22409436
2	Selssabil **KHELALFA**	22403233
3	Oumar **BAH**	22409261
4	Thierno Hammady **DIALLO**	22409301

Chargé de TP : M. Alexis Lechervy

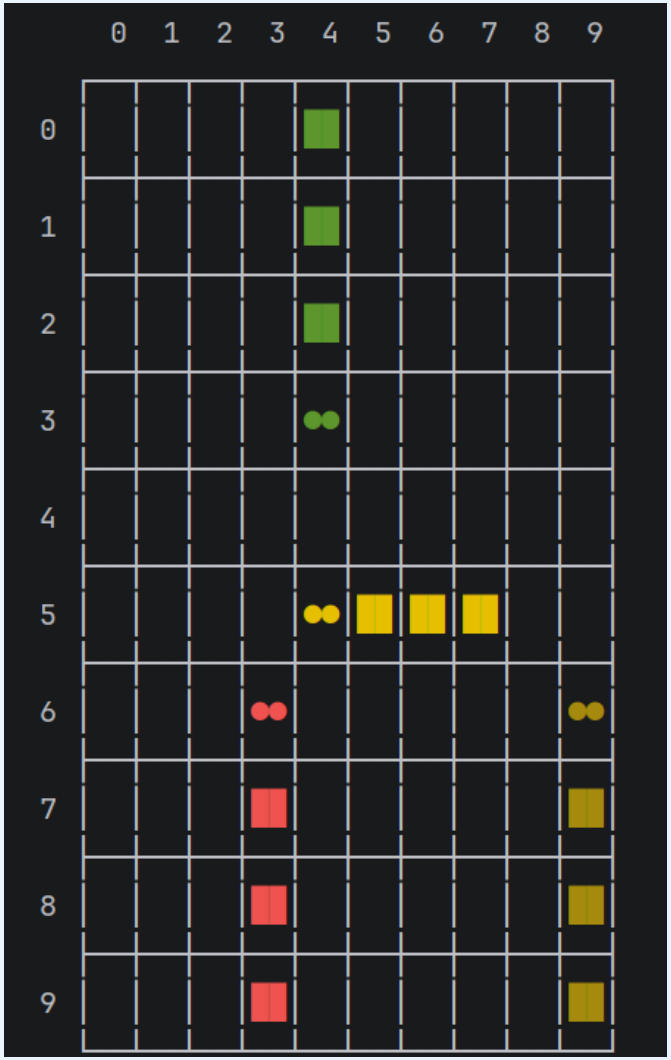
Présenté le 3 avril 2026

Plan de la présentation

1. Contexte du projet
2. Problématique et objectifs du projet
3. Architecture et modélisation
4. Répartition des tâches réalisées
5. Technologies et outils utilisés
6. Démonstration du jeu
7. Stratégies d'intelligence artificielle
8. Analyse et expérimentation
9. Conclusion et perspectives

Présentation du projet

- Tron est une variante multi-joueurs du jeu du serpent.
- Les joueurs se déplacent sur une grille et laissent derrière eux une trace qui devient un obstacle.
- Tous les déplacements modifient l'espace disponible pour les autres joueurs.
- Le but est de survivre plus longtemps que les autres, seul ou en équipe.
- Ce cadre est intéressant pour étudier des comportements multi-agents et des décisions automatiques.



Problématique et objectifs

Problématique

Comment comparer des stratégies de décision dans un jeu de Tron multi-joueurs, en tenant compte des équipes, des heuristiques et de la profondeur de recherche ?

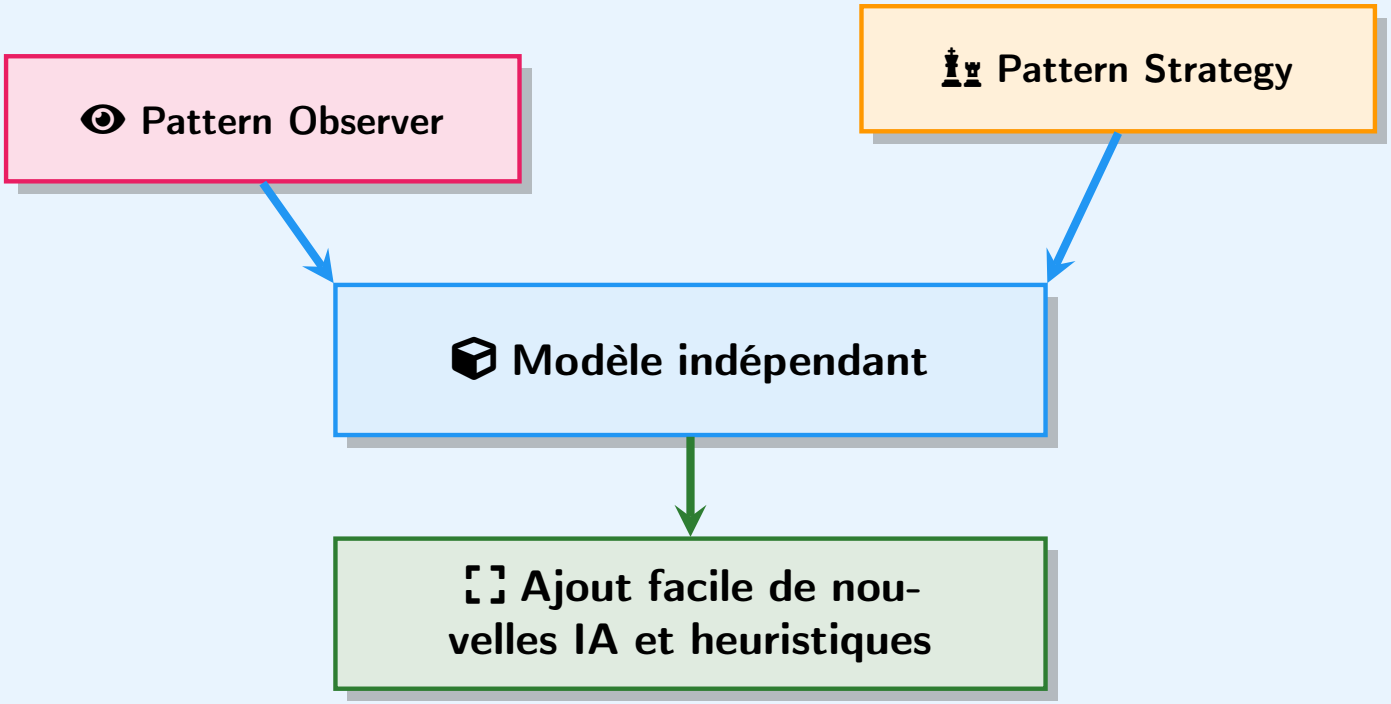
Objectifs du projet

- Construire un jeu de Tron multi-joueurs jouable automatiquement.
- Mettre en place plusieurs stratégies d'intelligence artificielle.
- Ajouter des heuristiques pour évaluer les positions.
- Lancer des expérimentations pour comparer les comportements obtenus.

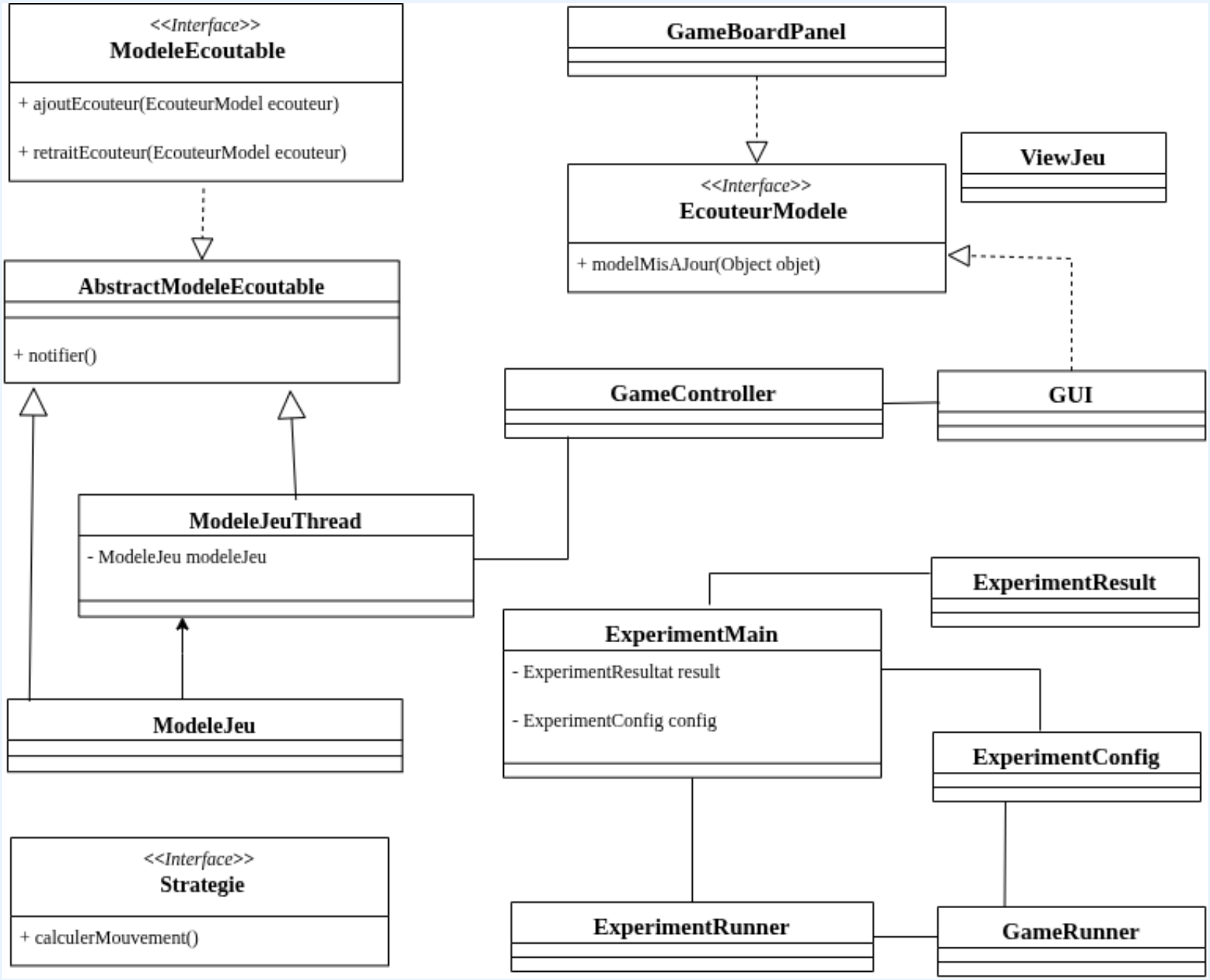
Idée générale de l'architecture

Concept clé

Le projet a été construit pour séparer clairement la logique du jeu, l'affichage et le contrôle de la partie. Cela permet de modifier une partie du système sans devoir tout réécrire.



Architecture MVC



Répartition des tâches

Développeur(se)	Implémentations réalisées
Oumar	• Modèle et cœur du jeu • Mise en place de l'architecture MVC • Participation à l'interface graphique
Lamia	• Pattern Strategy • Classes de test • UML et script d'expérimentation
Selssabil	• Interface graphique • Contrôleur et orchestration du jeu • Diagrammes d'analyse
Hammady	• Modèle et logique de jeu • Heuristiques et évaluateurs • Analyse et expérimentation des stratégies

Technologies et outils utilisés

Développement

Java 17

Swing

Conception & doc

UML 2.5

Draw.io

Versionnement

Git

GitHub

Build & scripts

Apache Ant

Bash / Shell

Analyse & tests

JFreeChart

JUnit 4

Rédaction

LaTeX

Beamer

Exemple d'exécution d'une partie avec l'interface graphique

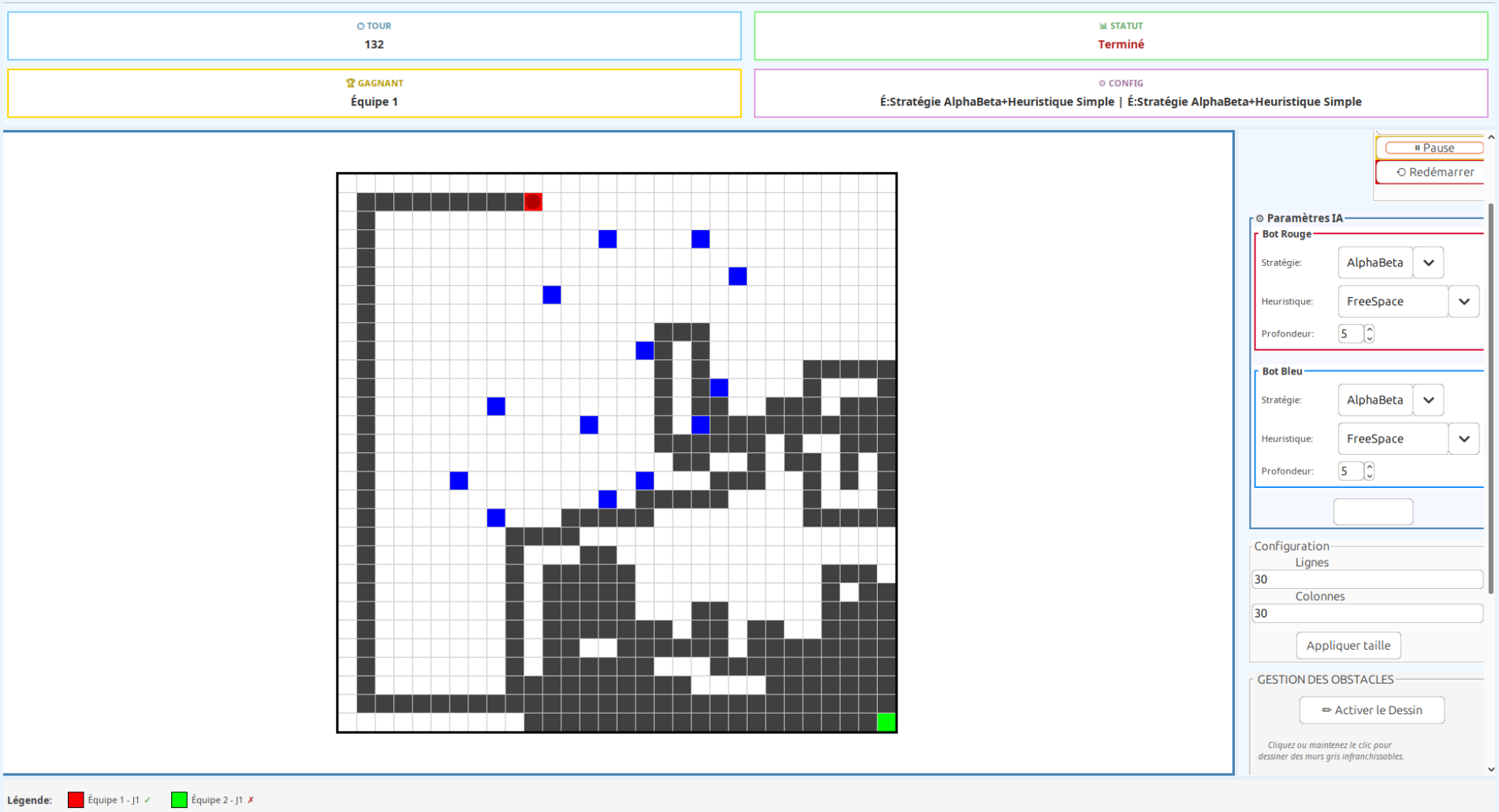
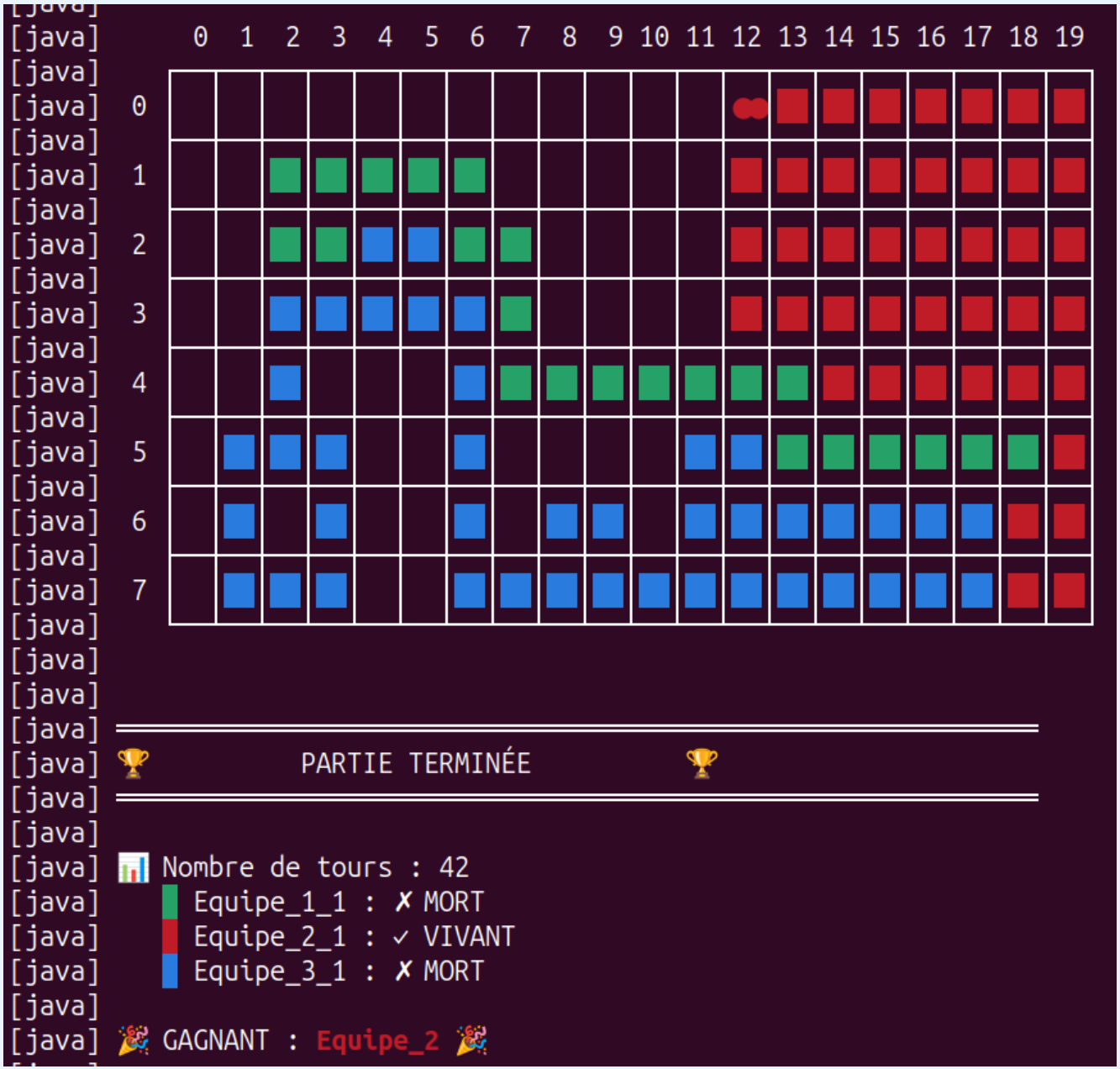


Figure 1 – Exemple d'exécution d'une partie avec l'interface graphique

Exemple d'exécution d'une partie avec l'interface console



Les 5 stratégies implémentées

Algorithmes de recherche

Pour 2 joueurs

MinMax

Alpha-Beta

Pour N joueurs

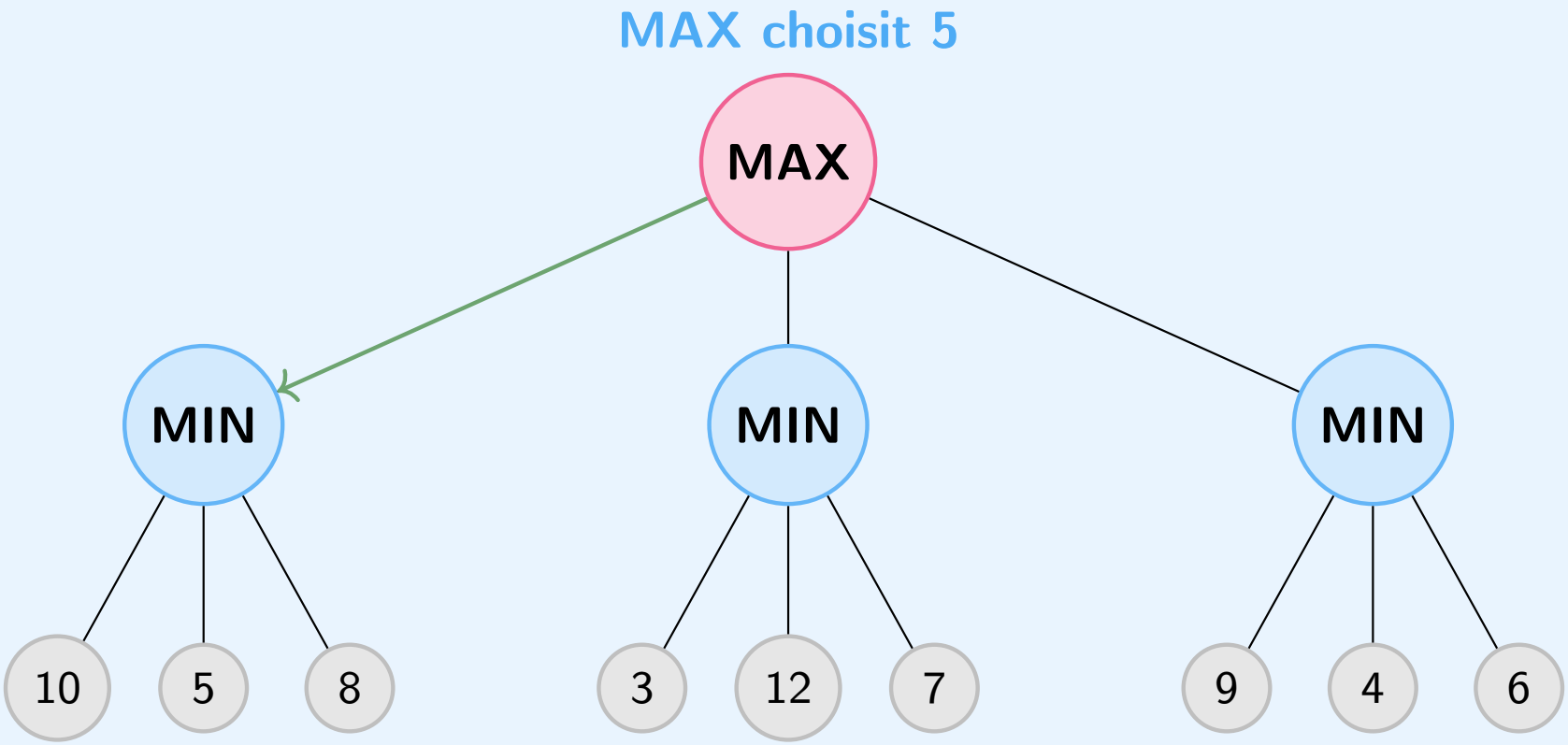
MaxN

Paranoid

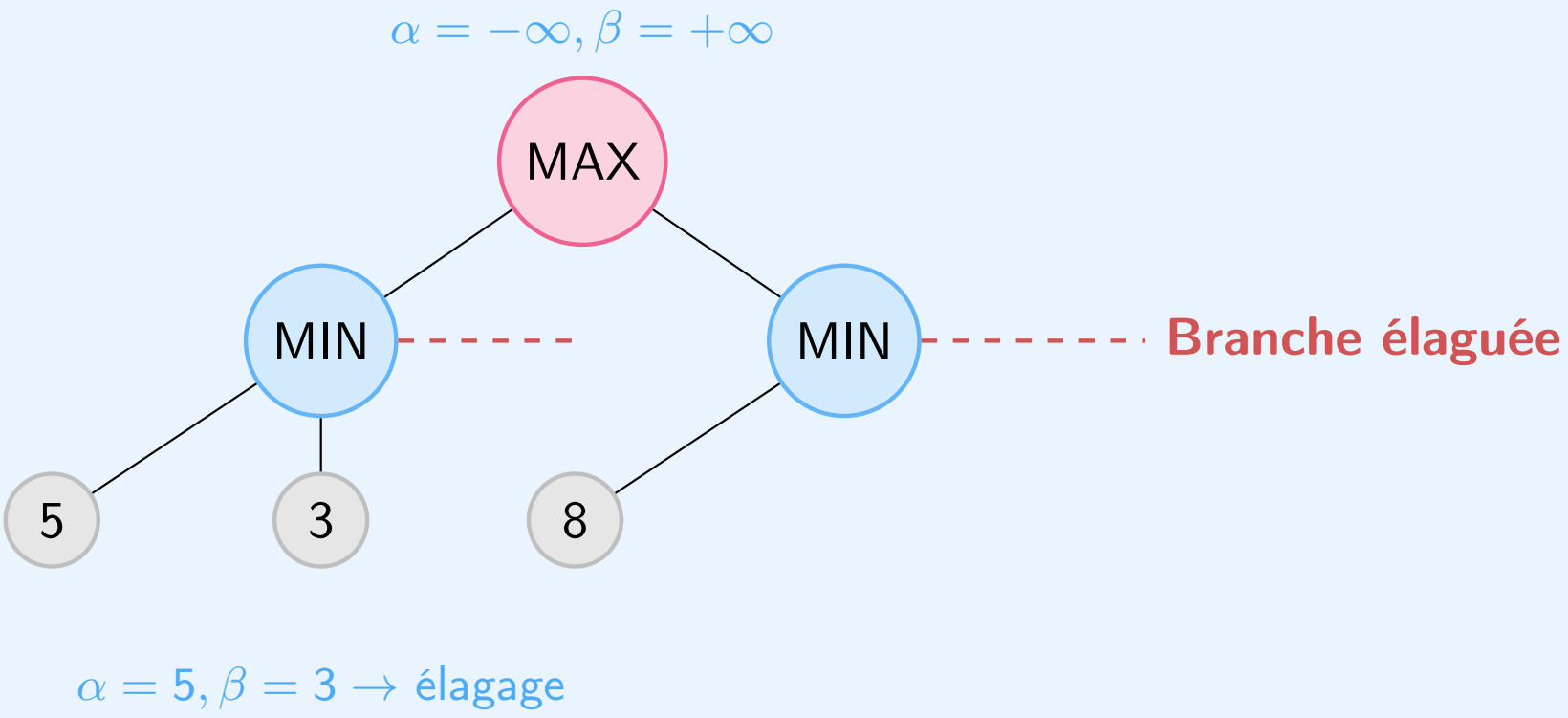
SOS

- **MinMax/Alpha-Beta : compétition à 2**
- **MaxN/Paranoid/SOS : extension à N joueurs**

MinMax - Schéma de l'arbre



Alpha-Beta - Principe de l'élagage



1. **Avantages :** Même résultat que *MinMax* mais beaucoup plus rapide
2. **Inconvénient :** Nécessite un bon ordre des coups. Si on explore les mauvaises trajectoires en premier, l'algorithme ne pourra pas couper de branches.

MaxN - Le vecteur de scores

Vecteur d'utilité : $[s_R, s_B, s_V]$

Score Rouge	Score Bleu	Score Vert
42	15	38
↓	↓	↓
Rouge maximise	Bleu maximise	Vert maximise

Principe MaxN

Chaque joueur maximise **sa propre case** dans le vecteur

MaxN vs Paranoid : deux logiques antagonistes

MaxN

Principe : Chacun maximise **son propre score**

- Vision **individualiste**
- Chaque joueur agit pour lui-même
- Comportement : *Égoïste*

> Je veux être le meilleur !

Paranoid

Principe : Tous les adversaires minimisent **MON score**

- Vision **pessimiste**
- Tous les adversaires sont en coalition
- Comportement : *Défensif*

> Ils sont tous contre moi !

SOS - Stratégie coopérative

Tous les joueurs reçoivent la même valeur !

SOS = utilité sociale identique

s_{Rouge}	s_{Bleu}	s_{Vert}
15 ↓	15 ↓	15 ↓

Principe SOS

Maximise le **bien collectif**, pas l'intérêt individuel.

Les 3 heuristiques implémentées

Heuristique	Principe	Coût
FreeSpace	Espace de manœuvre restant	Faible
Voronoi	Territoire contrôlé	Moyen
TreeOfChambers	Structure des espaces libres	Élevé

Simple $\succ$ Moyen $\succ$ Complexe

Précision croissante $\rightarrow$

Impact sur les algorithmes

Une bonne heuristique peut changer complètement le classement des stratégies !

Protocole expérimental retenu

Duel
2 équipes

10/15/20
taille

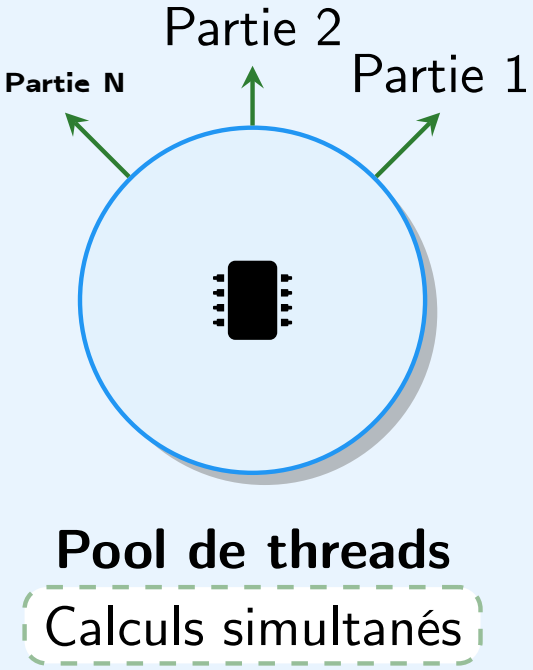
3/6/9
profondeur

50 parties
par config

Des positions aléatoires
protocole en duel pour plus de lisibilité

Multithreading et automatisation

- **Pool de threads** dans le mode expérimentation
- Plusieurs parties en **parallèle**
- **MaxN** : parallélisme à la racine
- Temps de calcul **réduit**



plusieurs configurations
en quelques heures !

Configuration et aperçu des résultats

Protocole

50 parties | Profondeur 3/6/9 | Grille 10/15/20

Question scientifique

Quelle influence la profondeur de recherche a-t-elle sur le jeu en fonction de la taille des équipes et de la grille ?

Impact de la profondeur de recherche

Résumé des Sessions (Confiance Globale)

Parties	T1	T2	T3	T4	Conf %
Config	ABS|VoH	MaxN|VoH	PaS|VoH	SOS|VoH	-
200	74.0%	9.0%	5.0%	4.5%	±6.06%
20	85.0%	5.0%	0.0%	5.0%	±16.25%
30	73.3%	3.3%	3.3%	6.7%	±15.34%
6	66.7%	0.0%	0.0%	0.0%	±30.59%
TOTAL	**74.6%**	**7.8%**	**4.3%**	**4.7%**	**±5.33%**

Résumé des Sessions (Confiance Detaillé)

Session	T1	T2	T3	T4
Nb Parties	ABS|VoH	MaxN|VoH	PaS|VoH	SOS|VoH
200	74.0% ±6.08%	9.0% ±3.97%	5.0% ±3.02%	4.5% ±2.87%
20	85.0% ±15.65%	5.0% ±9.55%	0.0% ±0.00%	5.0% ±9.55%
30	73.3% ±15.82%	3.3% ±6.42%	3.3% ±6.42%	6.7% ±8.93%
6	66.7% ±37.72%	0.0% ±0.00%	0.0% ±0.00%	0.0% ±0.00%
TOTAL GLOBAL	**74.6% ±5.33%**	**7.8% ±3.29%**	**4.3% ±2.48%**	**4.7% ±2.59%**

On a : L'équipe dominante est Equipe_1 avec 74,6% de victoires.

Impact de la profondeur de recherche

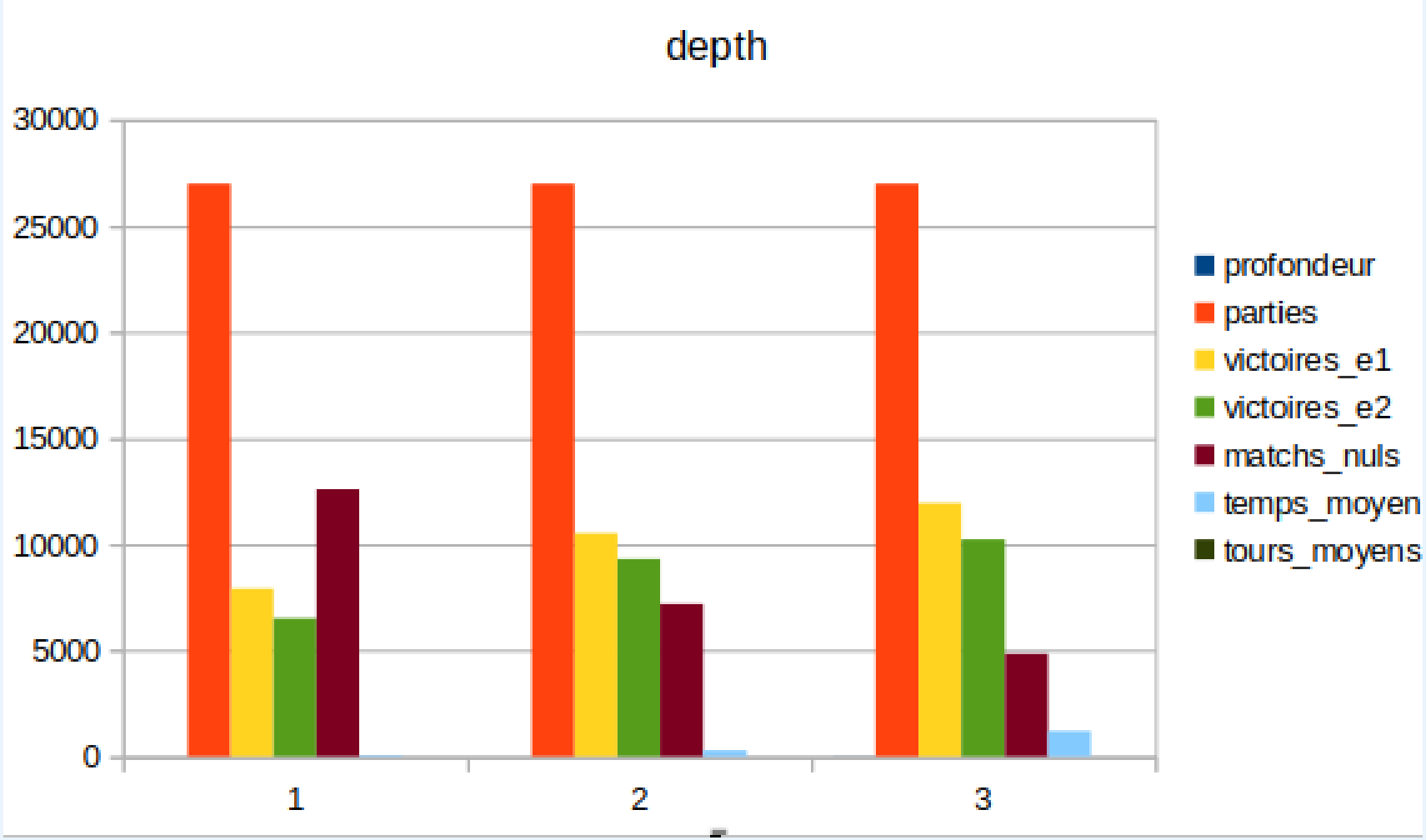


Figure 4 – Impact de la profondeur de recherche.

Comparaison des stratégies

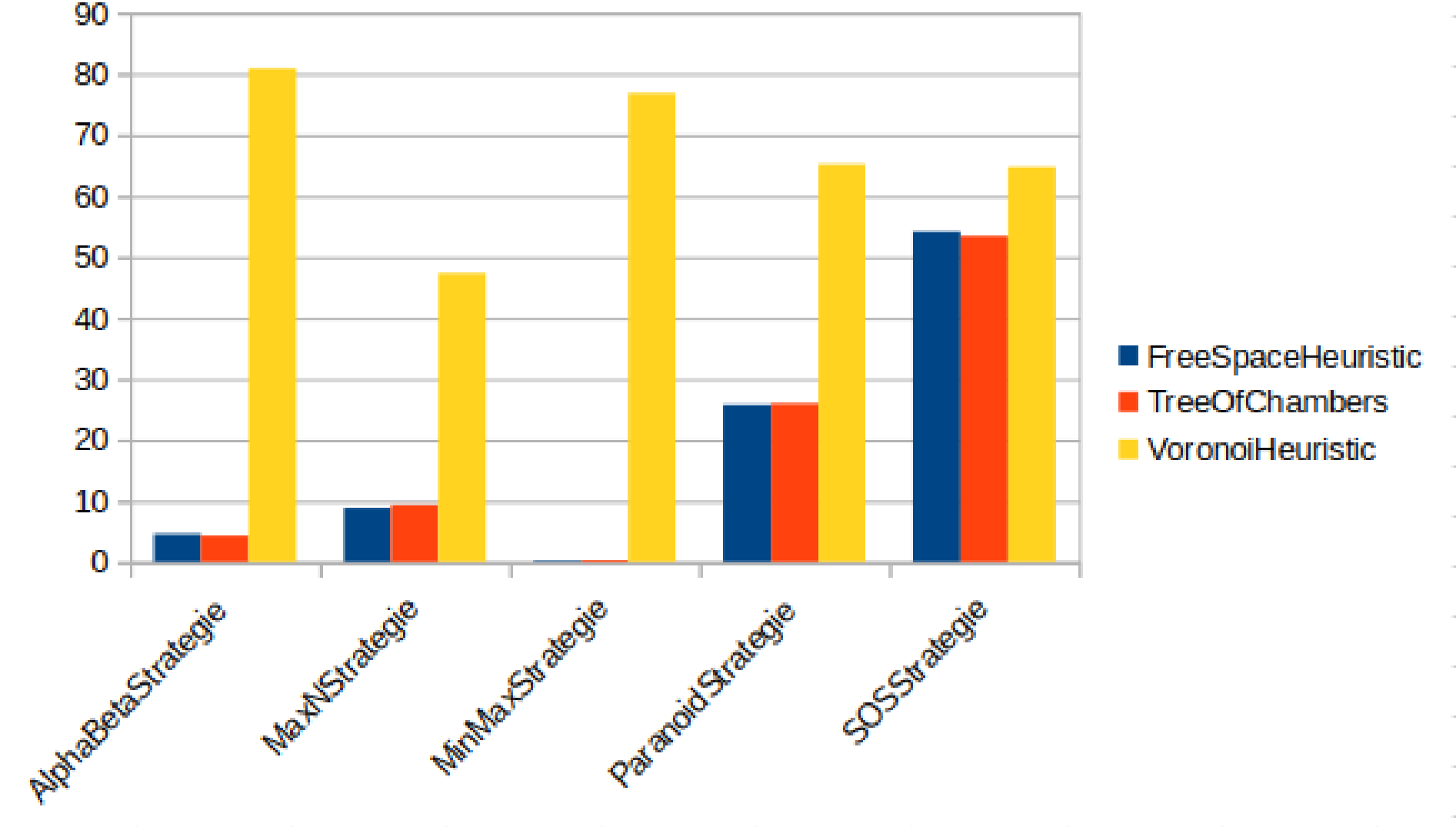


Figure 5 – Comparaison globale des stratégies avec les heuristiques

Tests principaux du projet

Heuristiques

- VoronoiHeuristic
- TreeOfChambersHeuristic
- FreeSpaceHeuristic

Stratégies

- SOSStrategie
- ParanoidStrategie
- MinMaxStrategie
- AlphaBetaStrategie

Modèle

- ModeleJeu

Objectifs atteints et difficultés rencontrées

Objectifs atteints

- Implémentation de trois heuristiques : Voronoi, TreeOfChambers et FreeSpace.
- Développement de cinq stratégies : SOS, Paranoid, MinMax, AlphaBeta et MaxN.
- Mise en place du modèle, de l'interface graphique et des tests principaux.
- Ajout d'un mode d'expérimentation avec export CSV et PDF.

Difficultés rencontrées

- Certaines stratégies deviennent coûteuses quand la profondeur augmente.
- La complexité du jeu multi-joueurs rend les comparaisons difficiles.
- Le choix du protocole expérimental n'a pas été simple à cause des positions de départ aléatoires.
- Le multithreading améliore les performances, mais demande aussi plus de vigilance dans l'organisation des simulations.

Perspectives d'amélioration

Piste possible et validation future

- Contrôle des positions de départ pour des comparaisons plus stables.
- Renforcement de l'analyse statistique des résultats.
- Extension du protocole expérimental à d'autres configurations.
- Optimisation des algorithmes de recherche.

Bilan du projet

Points forts

- Une architecture modulaire qui facilite l'ajout de nouvelles stratégies.
- Une diversité d'approches IA pour comparer plusieurs comportements.
- Un projet complet avec interface, expérimentation, export et tests.

Apports du projet

- Meilleure compréhension des algorithmes de recherche multi-agents.
- Travail en équipe avec le système de versioning Git.
- Première approche d'une analyse expérimentale sur un jeu compétitif.

Ce projet nous a permis de relier conception logicielle, intelligence artificielle et expérimentation autour du jeu de Tron.

Merci pour votre attention !

Questions ?

Suggestions !